ICS 2026 Bomb Lab 实验报告

姓名：黄靖修

学号：26303050101

完成日期：2026 年 10 月 28日

实验状态：全部完成

1. 实验概述与验证记录

本报告按 Phase 1、2、3、4、5、6、Honor Phase 的顺序整理已完成关卡，并依据实验文档 bomb.pdf 以及 GDB 实际分析结果恢复关键汇编与计算关系。

公开测试结果：前六关 60/60，Honor public tests 10/10。

非公开列表 seed：314159265。该 seed 对每一关及隐藏关均完成了一次端到端验证：

- Phase 1：challenge 0xdddd3a7d5f62eab2f，显示 Phase 1 defused。
- Phase 2：challenge 0x2d7241e89d8558ec，显示 Phase 2 defused。
- Phase 3：challenge 0xc8f302267d8e98d7，显示 Phase 3 defused。
- Phase 4：challenge 0x8697010d52ce87c5，显示 Phase 4 defused。
- Phase 5：challenge 0x28999ba079ba4216，显示 Phase 5 defused。
- Phase 6：challenge 0x72b2b143b0462164，显示 Phase 6 defused。
- Honor phase：同一 seed 下显示 Secret phase defused 和附加档案已恢复。

所有 challenge 与结果均来自本次实际运行截图，没有使用公开测试 seed 列表中的值代替。

2. Phase 1：拆分与多位运算

2.1 关键汇编与数据流

Phase 1 将一个 64 位 challenge 拆为高 32 位、低 32 位和右移 16 位的值，再依次调用三个子函数。关键语义对应 shr、imul、xor 和 add 等操作。

```
shr $32, high32
mov challenge, low32
shr $16, mid32
; step1、step0、step2 通过三次函数调用串联
```

数据流为：64 位 challenge → 高/低/中段 → step1 → step0 → step2 → 无符号十进制输出。

2.2 恢复出的计算关系

```
uint32_t high32 = (uint32_t)(challenge >> 32);
uint32_t low32  = (uint32_t)challenge;
uint32_t mid32  = (uint32_t)(challenge >> 16);

uint32_t step1_ret = (high32 * 3u) ^ (mid32 + 0x0fddu);
uint32_t step0_ret = (high32 ^ 0x0553u) + (low32 * 8u);
uint32_t answer =
    (step0_ret + step1_ret * 8u) ^ 0x7245u;
```

所有计算均按 32 位无符号整数回绕。输入只有一个十进制无符号整数。

2.3 推理与验证

先确认 64 位参数的拆位方式，再根据三个辅助函数的结果依赖关系确定调用顺序。使用 challenge 直接代入公式即可生成答案，不需要枚举。

额外 seed 314159265 下，Phase 1 challenge 为 0xdddd3a7d5f62eab2f，程序显示 Phase 1 defused.。

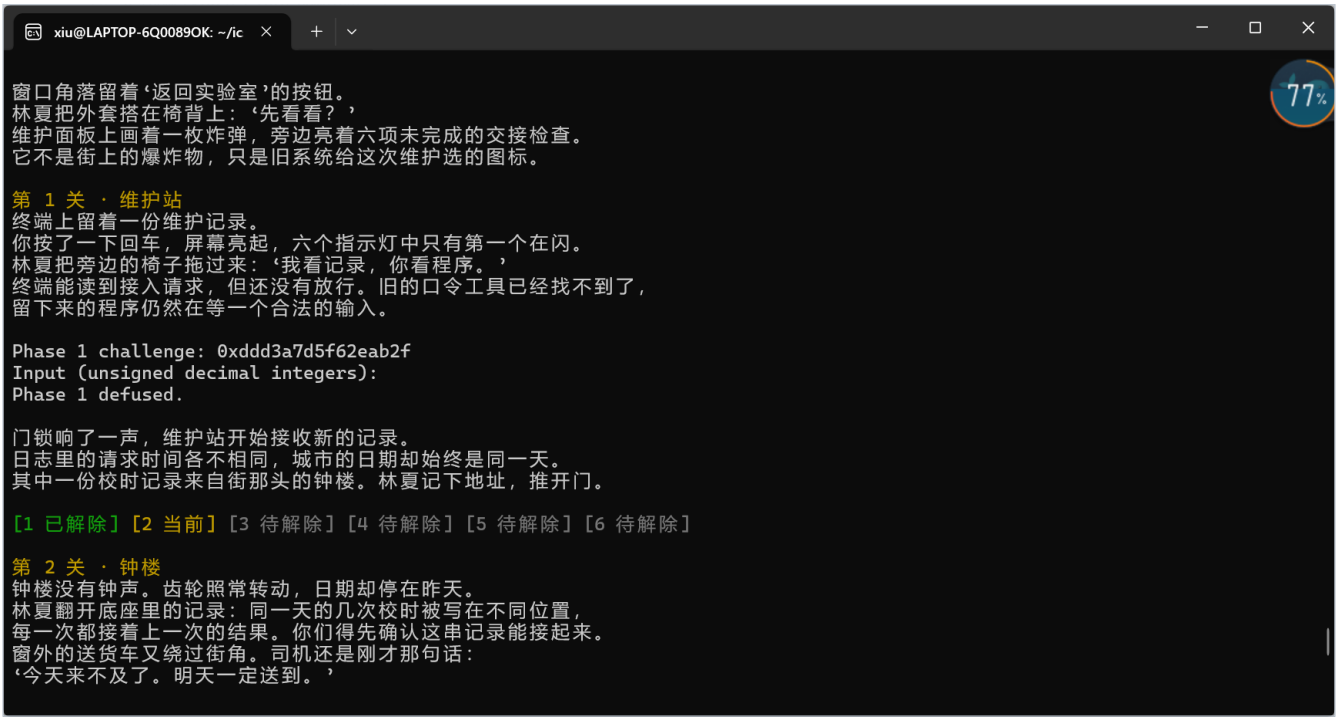


图 1　非公开列表 seed 314159265：Phase 1 通过。

3. Phase 2：递推序列与位截取

3.1 关键汇编

程序用 sscanf 解析 6 个整数。随后进行 5 次循环，每次通过 shr 定位 challenge 的一段，通过 and $0xf 取出 4 位，再与前一结果异或并乘以 5。

```
call sscanf
shr  ; 根据循环编号定位位段
and  $0xf, %reg
xor  %prev, %reg
imul $5, %reg
```

3.2 恢复出的计算关系

```
x[0] = ((challenge >> 8) & 0x7fu) * 3u;

for (unsigned int i = 1; i <= 5; ++i) {
    uint32_t shift = 11u + (i - 1u) * 3u;
    uint32_t chunk = (uint32_t)(challenge >> shift) & 0xfu;
    x[i] = ((chunk + i) ^ x[i - 1]) * 5u;
}
```

最终输出六个十进制无符号整数。每一位的新结果都依赖前一位，因此循环必须严格按 i = 1..5 执行。

3.3 推理与验证

首个字段只使用 challenge 的第 8 到第 14 位；后续字段使用 4 位片段，并递推得到序列。位宽、循环次数和乘法回绕是该关的三个关键点。

额外 seed 314159265 下，Phase 2 challenge 为 0x2d7241e89d8558ec，程序显示 Phase 2 defused.。



图 2　非公开列表 seed 314159265：Phase 2 通过。

4. Phase 3：Switch-Case 跳转表与乱序映射

4.1 关键汇编

输入为两个整数。challenge & 0x7 作为索引，通过跳转表间接跳转：

```
challenge &amp;amp; 0x7
jmp *0x405820(,%rax,8)
```

跳转表目标被打乱，不能假设索引与 case 编号一一对应。GDB 检查 0x405820 处的指针后发现索引 0 对应 case 7 的逻辑，索引 1 对应 case 1。

4.2 恢复出的计算关系

```
uint32_t index = (uint32_t)challenge &amp; 0x7u;
uint32_t val =
    (uint32_t)(challenge >> 3) ^
    (uint32_t)(challenge >> 32);
```

以 case 0 为例，其返回关系为：

```
return 0x0b391u + val * 3u;
```

其余分支根据跳转表索引逐个恢复；最终输出两个十进制无符号整数。

4.3 推理与验证

该关的核心风险是“代码块编号”和“表索引”不一致。正确方法是先用 GDB 读取 8 个跳转表项，再按索引对应的代码块翻译，而不是按源代码中 case 的顺序猜测。

额外 seed 314159265 下，Phase 3 challenge 为 0xc8f302267d8e98d7，程序显示 Phase 3 defused.。

图 2 同时包含 Phase 3 的 challenge 与通过记录。

5. Phase 4：递归与二分查找模拟

5.1 关键汇编

输入为一个整数。主函数调用 p4_walk，其中存在对自身的 call p4_walk，并根据当前中值与目标 key 的大小关系选择左右区间。

```
call p4_walk
; ret = p4_walk(...)
imul $5, ret
xor  temp, ret
xor  $0x14 / $0x1d, ret
```

数据流为：key 与初始区间 → 计算 mid → 比较 → 二分递归 → 返回路径逐层变换 → 得到答案。

5.2 恢复出的计算关系

```
uint32_t key = (uint32_t)challenge & 0x7fu;
uint32_t left = 0;
uint32_t right = 0x7fu;
uint32_t x =
    (uint32_t)(challenge >> 7) ^
    (uint32_t)(challenge >> 32);

mid = left + ((right - left) >> 1);

if (key == mid) {
    temp = (mid + 7u) ^ x;
} else if (key &gt; mid) {
    ret = p4_walk(key, mid + 1u, right, x);
    ret = ((ret * 5u) ^ temp) ^ 0x14u;
} else {
    ret = p4_walk(key, left, mid - 1u, x);
    ret = ((ret * 5u) ^ temp) ^ 0x1du;
}
```

递归返回时需要保留当前 mid 生成的掩码 temp，否则无法正确还原最终值。

5.3 推理与验证

该关等价于在大小为 127 的区间上模拟二分查找。判断递归方向时使用 key 与 mid 的大小关系，返回时则按左/右分支分别执行异或常数。递归层数和每层 temp 的计算是主要难点。

额外 seed 314159265 下，Phase 4 challenge 为 0x8697010d52ce87c5，程序显示 Phase 4 defused.。



图 3　非公开列表 seed 314159265：Phase 4 通过。

6. Phase 5：两字段校验与链表遍历

6.1 输入与成功条件

phase_05 先调用 valid_u32_fields(input, 2)，然后使用地址 0x403186 处的格式字符串解析输入。GDB 读取结果为 %u %u。

关键控制流如下：

```
call valid_u32_fields
call sscanf(input, "%u %u", &a, &b)
cmp  $2, %eax
jne  fail

lea  -0x14(%rbp), %rsi
mov  %r12, %rdi
call p5_traverse

cmp  %edx, -0x1c(%rbp)
cmp  %eax, -0x18(%rbp)
```

因此输入包含两个无符号十进制整数。第一个必须等于 p5_traverse 的返回值，第二个必须等于 p5_traverse 写入输出指针的节点 value。

数据流为：输入字符串 → valid_u32_fields → sscanf → p5_traverse → 两次比较 → 接受或 BOOM。

6.2 p5_traverse 的数据结构与公式

表头位于 0x405b60，共 8 个链表头。每个节点占 16 字节：next 位于 +0x00，value 位于 +0x08，salt 位于 +0x0c。

关键汇编如下：

```
mov 0x405b60(,%rax,8), %rdx   # heads[key & 7]
shr $3, %rcx                  # steps = (key >> 3) & 3
shr $6, %rax
shr $0x20, %rdi
xor %edi, %eax
xor $0xfc32, %eax
add 0xc(%rdx), %eax
lea (%rax,%rax,4), %eax       # ×5
xor 0x8(%rdx), %eax
```

设 K 为 Phase 5 key，所有运算均按 32 位无符号整数回绕，恢复出的关系为：

```
idx   = K & 7;
steps = (K >> 3) & 3;
x     = ((uint32_t)(K >> 6) ^ (uint32_t)(K >> 32)) ^ 0xfc32u;

for (i = 0; i <= steps; ++i) {
    x = 5u * (x + node.salt);
    *out = node.value;
    x ^= node.value;
    node = node.next;
}
```

最终输出为 x out。

6.3 关键推理与纠错

最初把节点 0x405c40 的打包数据读成 0x0000f4600000027b，导致部分 seed 失败。用同一 challenge 对比 GDB 中 p5_traverse 的返回值与 C 程序输出后，发现该节点的 salt 应为 0x45f6，正确打包值为 0x000045f60000027b。

对 seed 0 对应的 challenge 0x53cb9f0c747ea2ea，GDB 和修正后的求解程序都得到：

```
2493607368 1396
p5_traverse return = 0x94a16dc8
```

该节点修正后，Phase 5 公开测试全部通过。

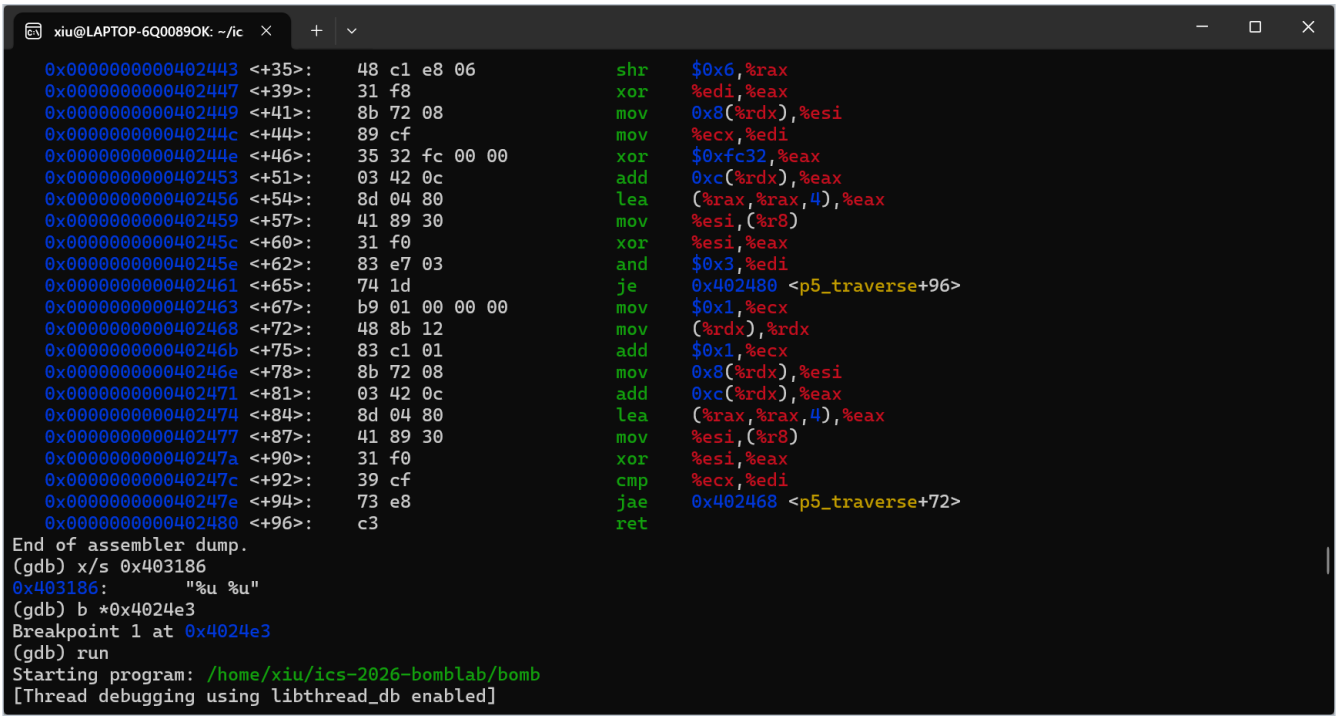


图 4　p5_traverse 的遍历、乘五和异或操作。

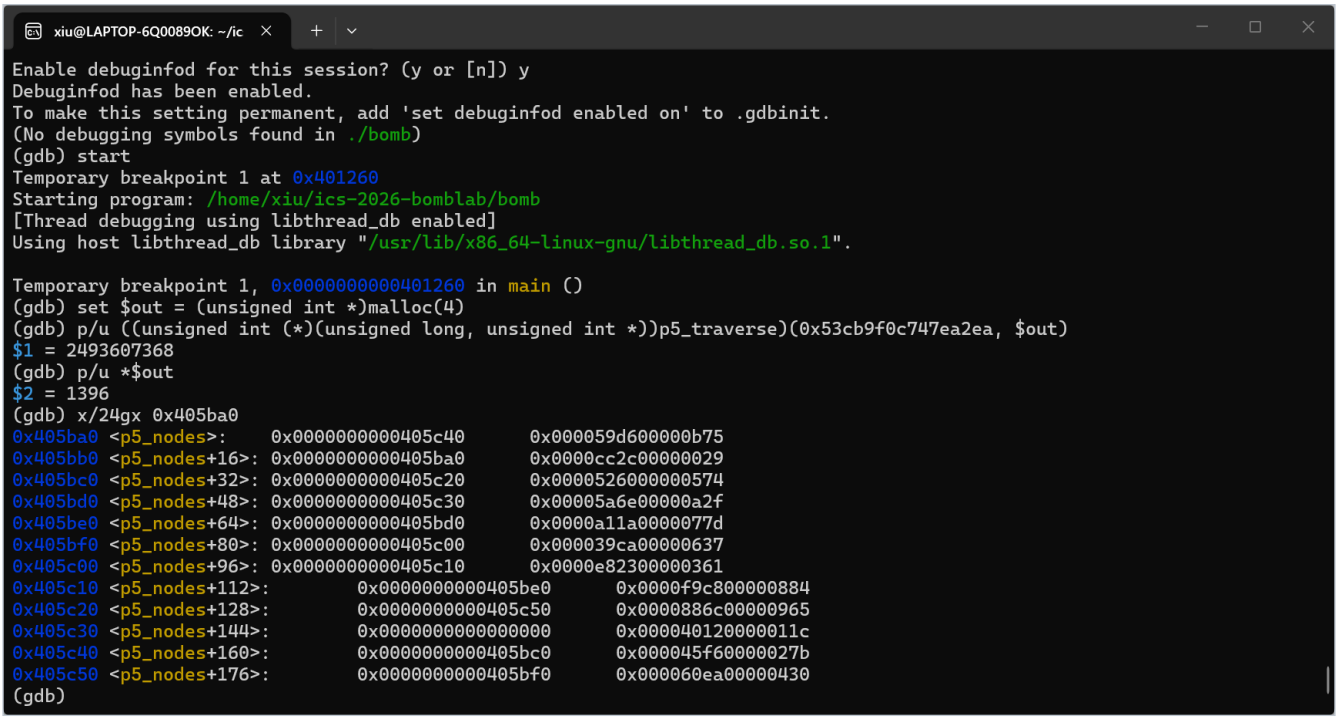


图 5　链表节点运行时数据，以及 GDB 中的标准返回值与输出字段。

6.4 运行与额外 seed 验证

公开测试中第 5 关相关检查全部通过。额外使用 seed 314159265。该 seed 下 Phase 5 challenge 为 0x28999ba079ba4216，程序显示 Phase 5 defused.。



图 6　公开阶段测试完成，总计数为 60/60。



图 7　非公开列表 seed 314159265：Phase 5 challenge 与 defused 输出。

7. Phase 6：双层函数指针分发

7.1 关键汇编

输入为一个整数。p6_dispatch 通过函数指针表调用 8 个底层运算函数：

```
call *0x405e00(,%rax,8)
```

主流程进行两次分发，两次索引和中间值均来自 challenge。

7.2 恢复出的计算关系

```
uint32_t arg1 =
    (uint32_t)(challenge >> 6) ^
    (uint32_t)(challenge >> 32);

uint32_t idx1 = (uint32_t)challenge & 0x7u;
uint32_t ret1 = p6_dispatch(arg1, idx1);

uint32_t res1 = ret1 ^ 0x89eeu;
uint32_t idx2 = (uint32_t)(challenge >> 3) & 0x7u;
uint32_t ret2 = p6_dispatch(res1, idx2);
uint32_t answer = ret2 + 0x663u;
```

8 个底层操作分别由 p6_op_0 到 p6_op_7 实现，内部包含移位、异或和乘法等 32 位运算。报告中只列出主分发关系，底层操作按函数指针表逐项翻译。

7.3 推理与验证

该关必须两次读取函数指针表：第一次确定对 arg1 使用哪个底层操作，第二次确定如何处理第一次结果。两次索引的位段分别是 challenge & 7 和 (challenge >> 3) & 7，不能混用。

额外 seed 314159265 下，Phase 6 challenge 为 0x72b2b143b0462164，程序显示 Phase 6 defused.。



图 8　非公开列表 seed 314159265：Phase 6 通过。

8. Honor Phase：隐藏入口与三字段验证

8.1 隐藏入口

secret_gate 调用 strlen 后减去 36，并检查结果不超过 127，因此隐藏输入总长度必须在 36 到 163 字节之间。随后比较缓冲区偏移 +32 处的四个字节，汇编指令为：

```
cmpl $0x4e45504f, -0xb0(%rbp)
```

立即数按小端序对应 ASCII 字符串 OPEN。完整隐藏输入结构为：

```
[32 个任意字符][OPEN][field1] [field2] [field3]
```

secret_phase 只解析 OPEN 后面的三个无符号十进制整数，格式为 %u %u %u。

控制流为：32 字节前缀 → OPEN（偏移 +32）→ sscanf 三字段 → 三段关系校验 → Honor passed。

8.2 两个隐藏参数的来源

调用关系为：

```
call phase_challenge(&options, 7)
call transcript_context(lines)
call secret_gate(input, A, B)
call secret_phase(input + 36, A, B)
```

求解器接口与测试代码一致：

```
int solve_secret(uint64_t challenge, uint64_t transcript,
                 char *out, size_t cap);
```

8.3 恢复出的计算关系

令 A 为 challenge，B 为 transcript，定义：

```
aLo = (uint32_t)A;
aHi = (uint32_t)(A >> 32);
bLo = (uint32_t)B;
bHi = (uint32_t)(B >> 32);
p = aLo ^ aHi;
q = bLo ^ bHi;
s = B & 3;

mix(x) = ((x >> 1) ^ 0xecd2u) + (x ^ 0x6b82u);

field1 = s;
field2 = mix(p ^ q) ^ (uint32_t)(B >> 16) ^ (s * 0x6b82u);
tail   = 0xecd2u + q + s + (p >> 1);
field3 = field2 ^ tail;
```

所有中间值均为 32 位无符号整数。最后三个项在汇编中使用 lea 和 add，因此需要先相加，再与 field2 异或；逐项异或是错误的。

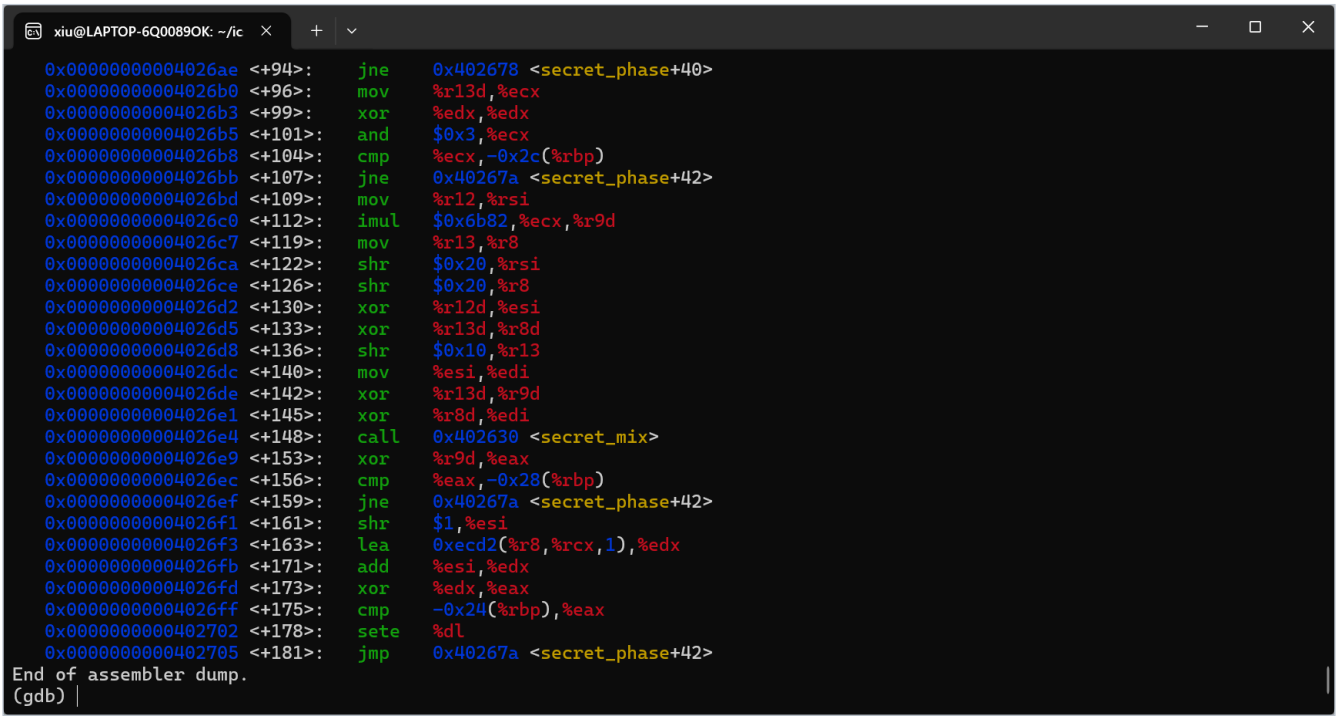


图 9　secret_mix 调用后字段二、字段三的计算。

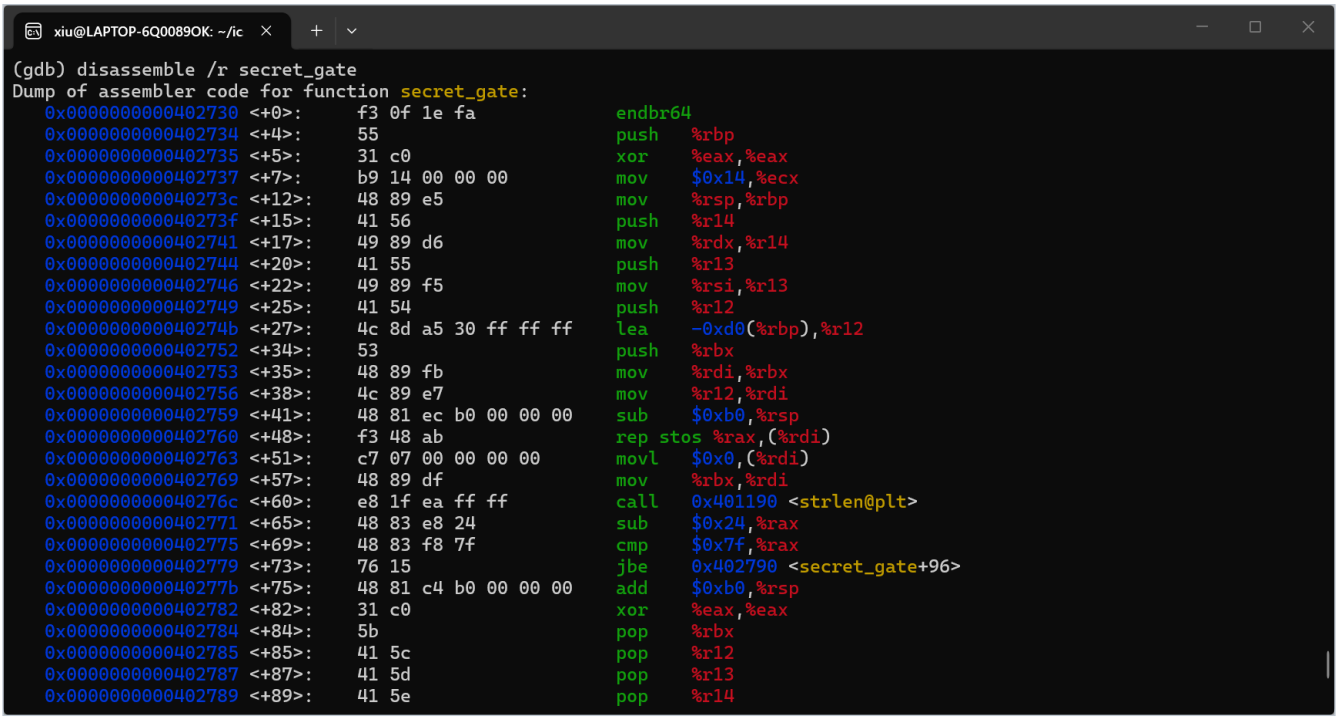


图 10　secret_gate 的长度检查、OPEN 判断和转交过程。

8.4 输出格式

测试框架先提供前六关答案，再把 defuser 的隐藏关输出追加为最后一行。因此 solve_secret 必须输出完整隐藏输入：

```
0123456789abcdef0123456789abcdefOPEN<field1> <field2> <field3>
```

其中前 32 个字节是任意前缀，紧接 OPEN。输出不能包含换行，并且必须是 NUL 结尾的非空字符串。

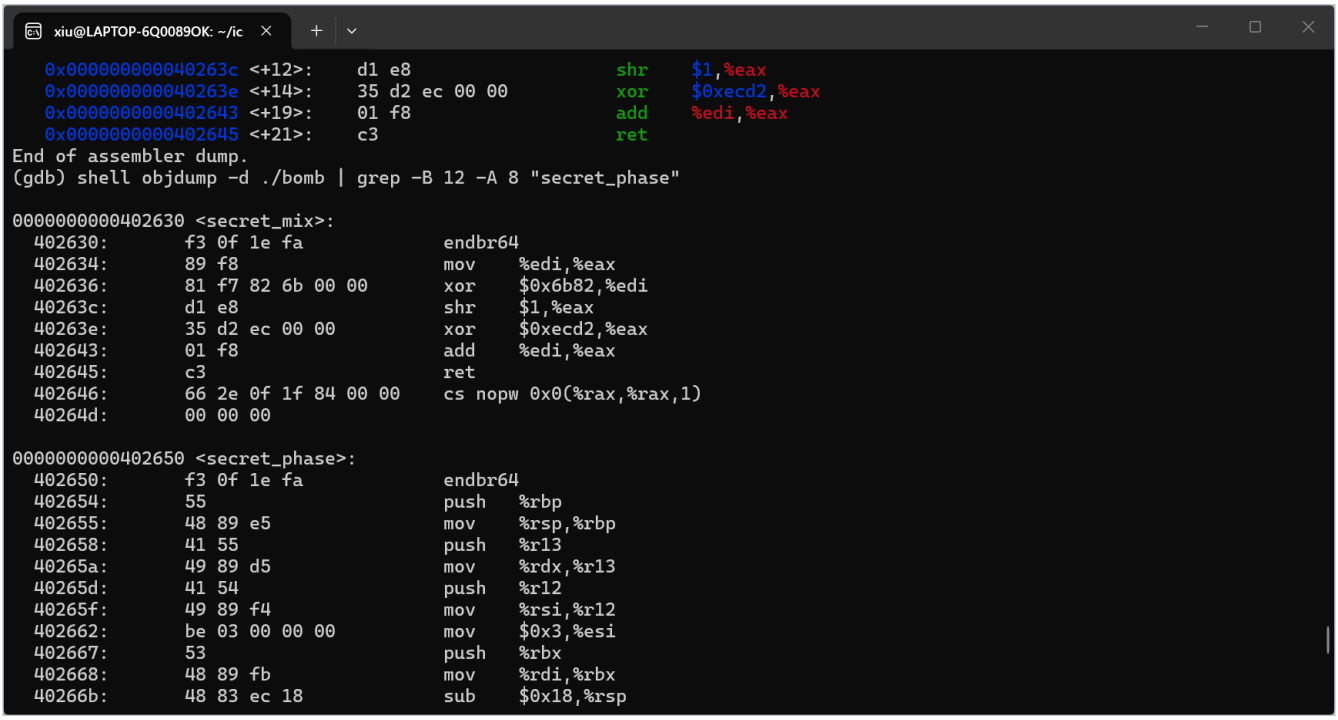


图 11　secret_phase 的三字段解析和第一个比较条件。

8.5 运行与额外 seed 验证

隐藏关公开测试最终为 10/10。使用非公开列表 seed 314159265 时，前六关依次显示 defused，随后显示：

> Secret phase defused.
> [SAFE] 城市照常运行。附加档案已恢复。

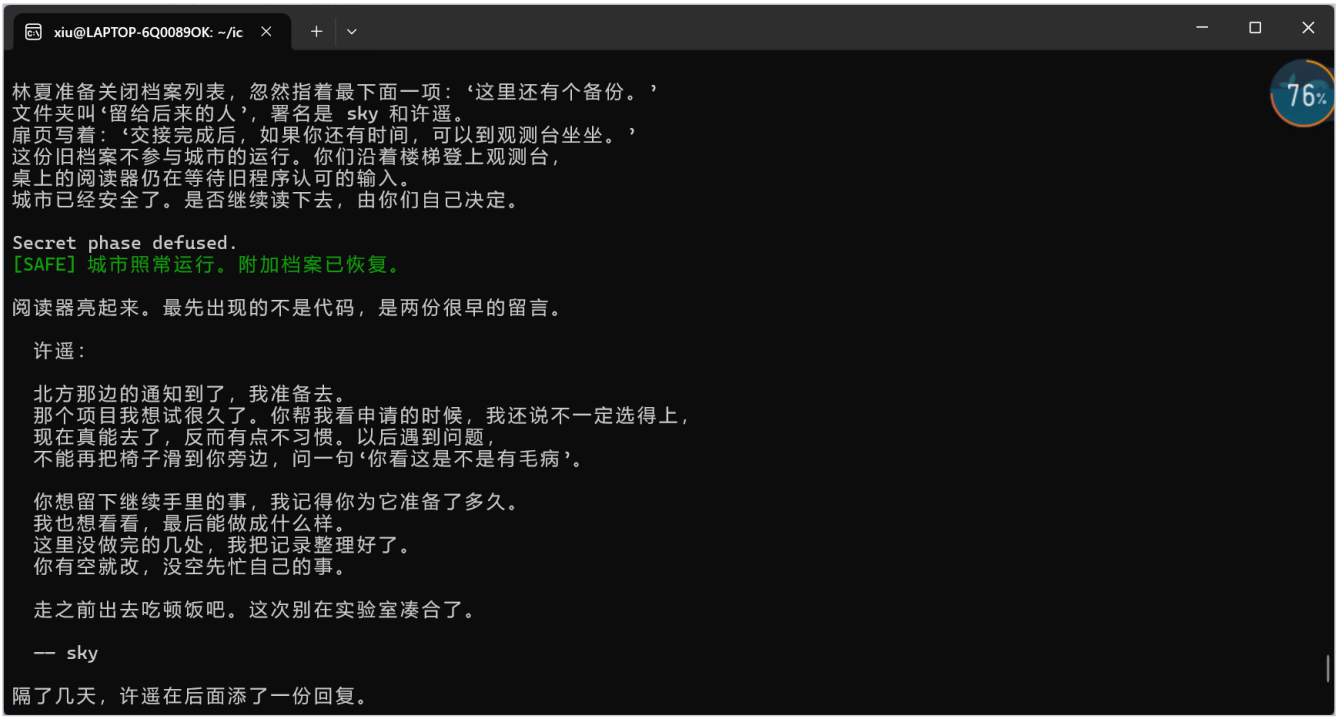


图 12　非公开列表 seed 314159265：Honor phase 成功，并显示 Secret phase defused。

9. 综合运行验证

非公开列表 seed 314159265 完整运行通过全部六关和隐藏关。公开阶段测试总计 60/60，Honor public tests 为 10/10。



图 13　全部阶段公开测试汇总：60/60。

10. 辅助工具与参考资料

10.1 工具用途

- GDB：用于反汇编、读取寄存器与内存、检查跳转表、函数指针表、链表节点和隐藏关参数。
- objdump：用于导出完整汇编及定位间接调用位置。
- VS Code：用于维护 defuser.c 和测试求解程序。
- defuser 与公开测试脚本：用于批量生成并验证各关答案。
- AI 编程助手 Codex：用于整理汇编语义、计算公式和报告结构；关键结果均由公开测试、额外 seed 和 bomb 实际运行复核。

10.2 参考资料

1. 课程提供的 bomb.pdf、项目 README、defuser.h 与 public_test.py。
2. CS:APP 课程 Bomb Lab 官方资料。
3. GDB 官方文档：https://sourceware.org/gdb/documentation/
4. GNU Binutils 文档：https://www.gnu.org/software/binutils/
5. System V AMD64 ABI：https://gitlab.com/x86-psABIs/x86-64-ABI
6. Bryant 与 O'Hallaron，Computer Systems: A Programmer's Perspective。

11. 实验体会与建议

本次实验的重点是依据实际二进制和控制流恢复关系，而不是猜测源码。64 位 challenge 与 32 位回绕、乱序跳转表、递归返回变换、链表和函数指针分发是最主要的易错点。对公开 seed 之外的 314159265 做完整验证，有助于确认实现不是针对测试数据硬编码。

建议课程材料继续强调 32 位回绕、跳转表可能乱序以及隐藏关完整输入格式；这些提示不会削弱分析难度，但能减少格式和整数宽度引起的非逻辑性错误。

附录

原始 bomb.pdf 材料

Bomb Lab 实验报告

一、 实验工具与引用资料
辅助工具：GDB（用于反汇编 disas、查看内存 x、设置断点 b 和单步调试）、objdump（用于导出完整汇编代码）、VS Code（用于编写 C 语言解答代码）。
参考资料：CS:APP 课程官方教材、GDB 官方文档、x86-64 汇编指令手册、b站。
二、 各关卡分析与推理过程
1. Phase 1: 拆解与多步位运算
关键汇编：主函数将 64 位的 challenge 拆分为高 32 位、低 32 位和右移 16 位。随后调用了 p1_step1、p1_step0、p1_step2 三个子函数进行按位异或、移位和乘法运算。
推理过程（计算关系）：
设 high32 = challenge >> 32, low32 = challenge & 0xFFFFFFFF, mid32 = challenge >> 16。
Step1: (high32 * 3) ^ (mid32 + 0xfdd)
Step0: (high32 ^ 0x553) + (low32 * 8)
Step2: (step0_ret + step1_ret * 8) ^ 0x7245
最终将计算结果转为无符号十进制整数输出。

2. Phase 2: 递推序列与位截取
关键汇编：sscanf 解析 6 个整数，进入一个循环。循环内通过 shr 和 and 0xf 提取 challenge 的碎片，并与前一个结果异或后乘以 5。
推理过程（计算关系）：
第一个数：x[0] = ((challenge >> 8) & 0x7F) * 3
后续 5 个数循环生成（设 i 为 1 到 5）：
提取位 chunk = (challenge >> (11 + (i-1)*3)) & 0xF
递推公式 x[i] = ((chunk + i) ^ x[i-1]) * 5

3. Phase 3: Switch-Case 跳转表与乱序陷阱
关键汇编：输入两个数字。主函数通过 challenge & 0x7 作为索引，利用 jmp *0x405820(,%rax,8) 进行间接跳转（Switch-Case 表）。
推理过程（计算关系）：
第一个数 x[0] = challenge & 0x7
第二个数的参数 val = (challenge >> 3) ^ (challenge >> 32)
重要发现：出题人对跳转表进行了乱序处理。通过 GDB 提取 0x405820 处的内存指针发现，索引 0 对应的是 case_7 的逻辑，索引 1 对应的是 case_1。严格按照乱序表映射进行 C 语言翻译（如 case 0: return (0xb391 + (val * 3))）。

4. Phase 4: 递归与二分查找模拟
关键汇编：输入 1 个整数。调用核心函数 p4_walk，该函数内部有 call p4_walk 自调用，且通过比较中值决定走左分支还是右分支。
推理过程（计算关系）：
这是一个模拟二分查找的递归函数。
初始参数：key = challenge & 0x7F, left = 0, right = 0x7F, x = (challenge >> 7) ^ (challenge >> 32)
递归逻辑：计算中点 mid = left + ((right - left) >> 1)。
若 key == mid，返回 (mid + 7) ^ x。
若 key > mid，递归 (key, mid+1, right, x) 后计算 ((ret * 5) ^ temp) ^ 0x14。
若 key < mid，递归 (key, left, mid-1, x) 后计算 ((ret * 5) ^ temp) ^ 0x1d。

6. Phase 6: 双层函数指针分发
关键汇编：输入 1 个整数。主函数调用 p6_dispatch，内部通过 call *0x405e00(,%rax,8) 进行两次函数指针表（共 8 个函数）的调用。
推理过程（计算关系）：
第一次分发：计算 arg1 = (challenge >> 6) ^ (challenge >> 32)，索引 idx1 = challenge & 0x7。调用得到 ret1。
中间运算：res1 = ret1 ^ 0x89ee。
第二次分发：索引 idx2 = (challenge >> 3) & 0x7。将 res1 作为参数再次调用 p6_dispatch 得到 ret2。
最终答案：ans = ret2 + 0x663。
通过 GDB 逆向这 8 个底层运算函数（如 p6_op_0 到 p6_op_5，其中包含移位、异或和乘法），严格翻译成 C 代码。

三、 对实验的意见和建议
本次实验极大地锻炼了我阅读 x86-64 汇编代码和熟练使用 GDB 进行动态调试的能力。尤其是第三关故意打乱跳转表的设置，让人印象深刻，体会到了逆向工程中必须 ‘所见即所得’，不能盲目相信源码推测。不过，部分关卡的随机 challenge 带来的计算溢出截断问题（如 Phase 5 的链表遍历），对 C 语言的底层强制类型转换提出了很高的要求，让我对汇编与高级语言的数据位宽对应有了更深的理解。”